

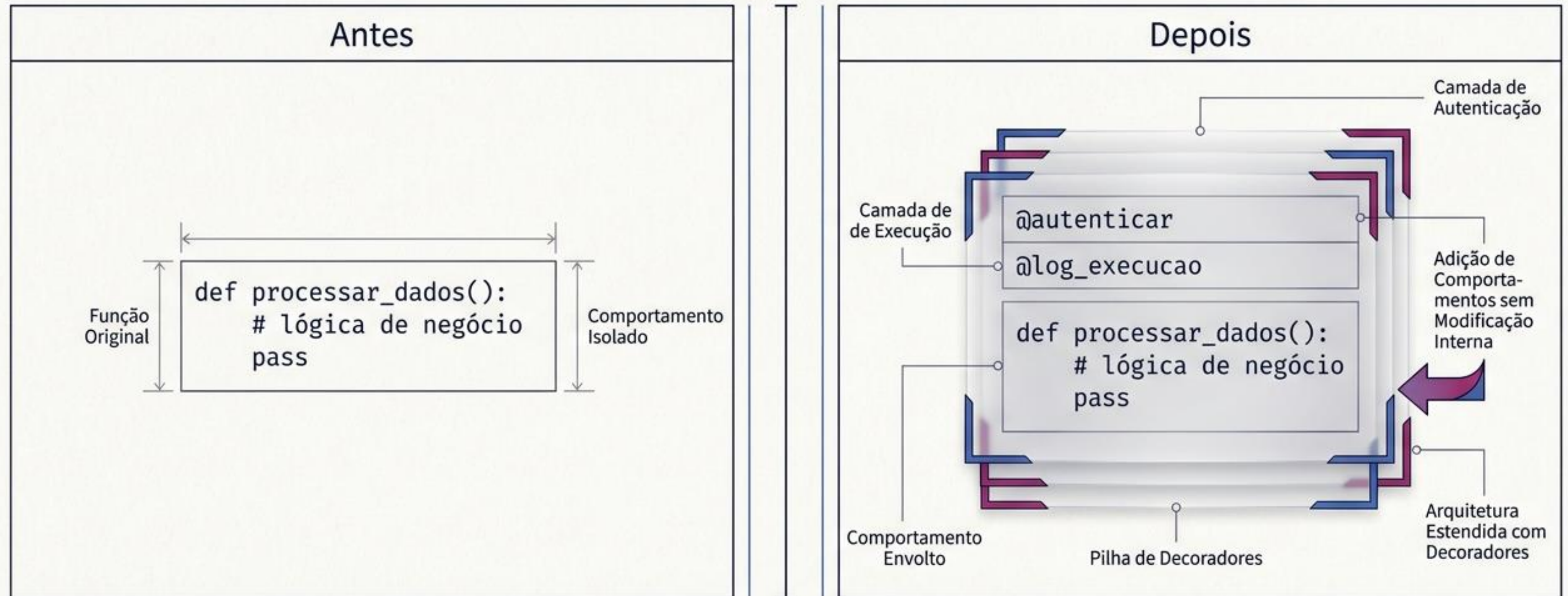
Domine Funções Decoradoras em Python

O Guia Definitivo: Mecânica, Aplicações e Melhores Práticas.



O “Truque de Mágica” da Arquitetura

Decoradores permitem alterar ou expandir o comportamento de uma função sem modificar seu código original. Parece mágica, mas é pura arquitetura.



O Problema: Lógica Transversal e Código Repetitivo

Violar o princípio DRY cria códigos frágeis e difíceis de manter. Como podemos isolar lógicas transversais (como logs, cache, transações de banco de dados e autorização) da regra de negócio principal?

```
class GestorUsuarios:
    def criar_usuario(self, usuario_atual, dados_novo_usuario):
        # Lógica transversal repetitiva
        tempo_inicio = time.time()
        print('Iniciando...')
        verificar_permissao(usuario_atual)

        # Lógica de negócio
        novo_usuario = Usuario(dados_novo_usuario)
        db.salvar(novo_usuario)
        print(f'Usuário {novo_usuario.id} criado.')

        # Lógica transversal repetitiva
        print(f'Tempo de execução: {time.time() - tempo_inicio:.2f}s')

    def deletar_usuario(self, usuario_atual, usuario_id):
        # Lógica transversal repetitiva
        tempo_inicio = time.time()
        print('Iniciando...')
        verificar_permissao(usuario_atual)

        # Lógica de negócio
        usuario = db.buscar_usuario(usuario_id)
        if usuario:
            db.deletar(usuario)
            print(f'Usuário {usuario_id} deletado.')

        # Lógica transversal repetitiva
        print(f'Tempo de execução: {time.time() - tempo_inicio:.2f}s')

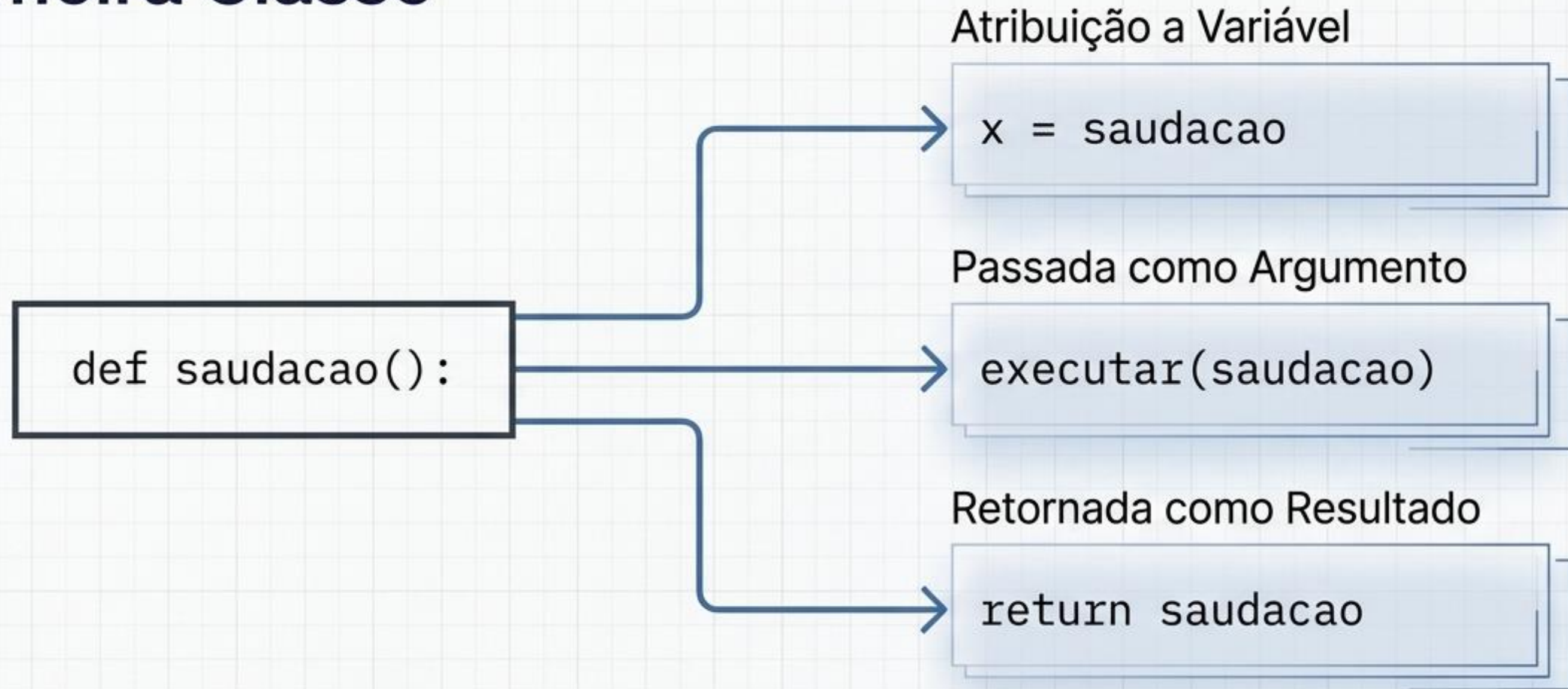
    def atualizar_usuario(self, usuario_atual, usuario_id, novos_dados):
        # Lógica transversal repetitiva
        tempo_inicio = time.time()
        print('Iniciando...')
        verificar_permissao(usuario_atual)

        # Lógica de negócio
        usuario = db.buscar_usuario(usuario_id)
        if usuario:
            usuario.atualizar(novos_dados)
            db.salvar(usuario)
            print(f'Usuário {usuario_id} atualizado.')

        # Lógica transversal repetitiva
        print(f'Tempo de execução: {time.time() - tempo_inicio:.2f}s')
```

Violar o princípio DRY cria códigos frágeis e difíceis de manter. Como podemos isolar lógicas transversais (como logs, cache, transações de banco de dados e autorização) da regra de negócio principal?

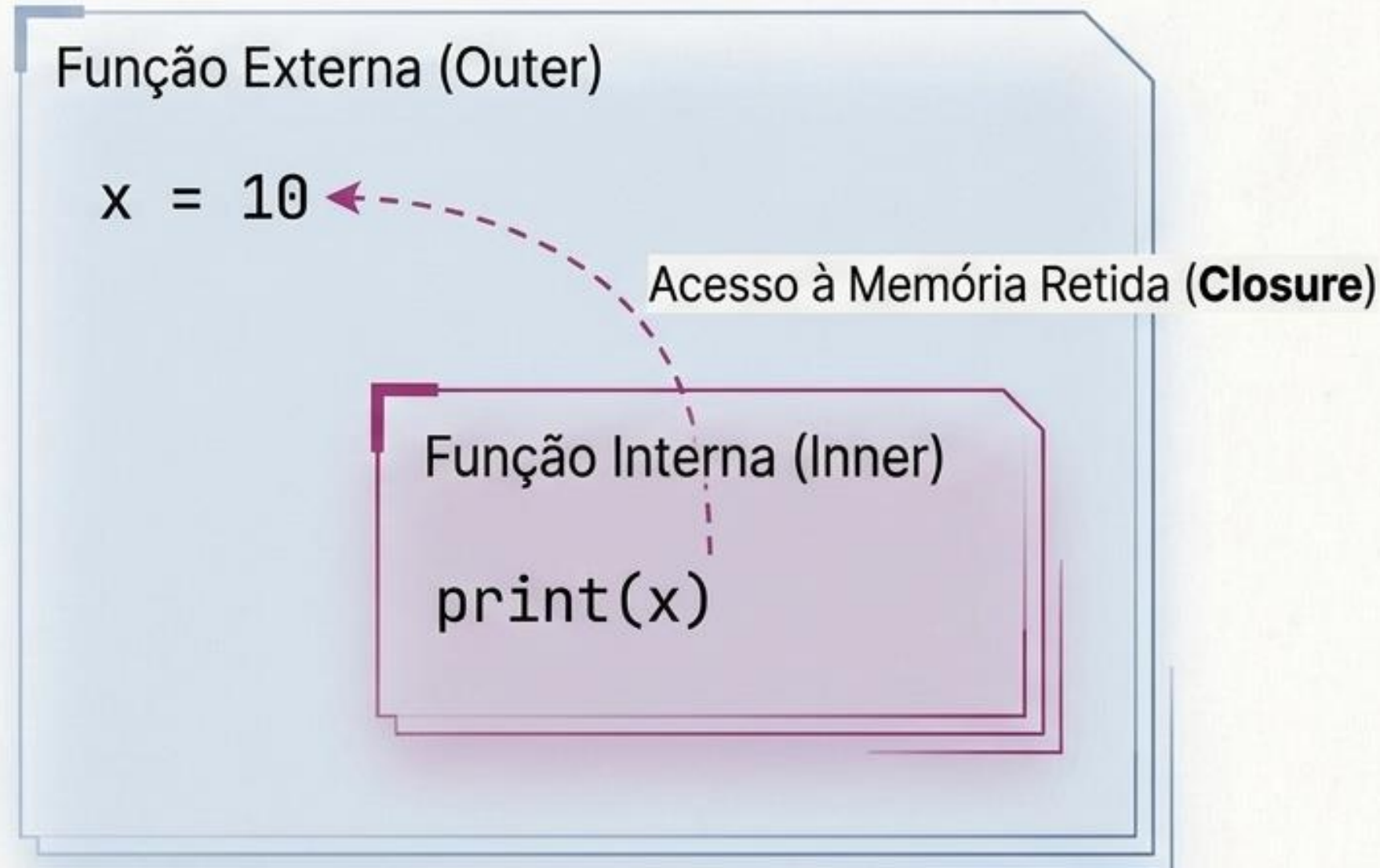
Fundamento 1: Funções como Cidadãos de Primeira Classe



Em Python, funções são objetos (instâncias da classe **function**). Elas podem ser passadas como argumentos, retornadas por outras funções e atribuídas a variáveis de forma dinâmica.

Fundamento 2: O Poder das Closures

Uma closure é uma função aninhada que “lembra” das variáveis de seu escopo externo, mesmo após a função externa ter concluído sua execução e saído da pilha (stack).



A Anatomia de um Decorador

A combinação de funções de primeira classe com closures cria o padrão do decorador. O uso de ***args**, ****kwargs** garante que o wrapper possa lidar com qualquer assinatura de função.

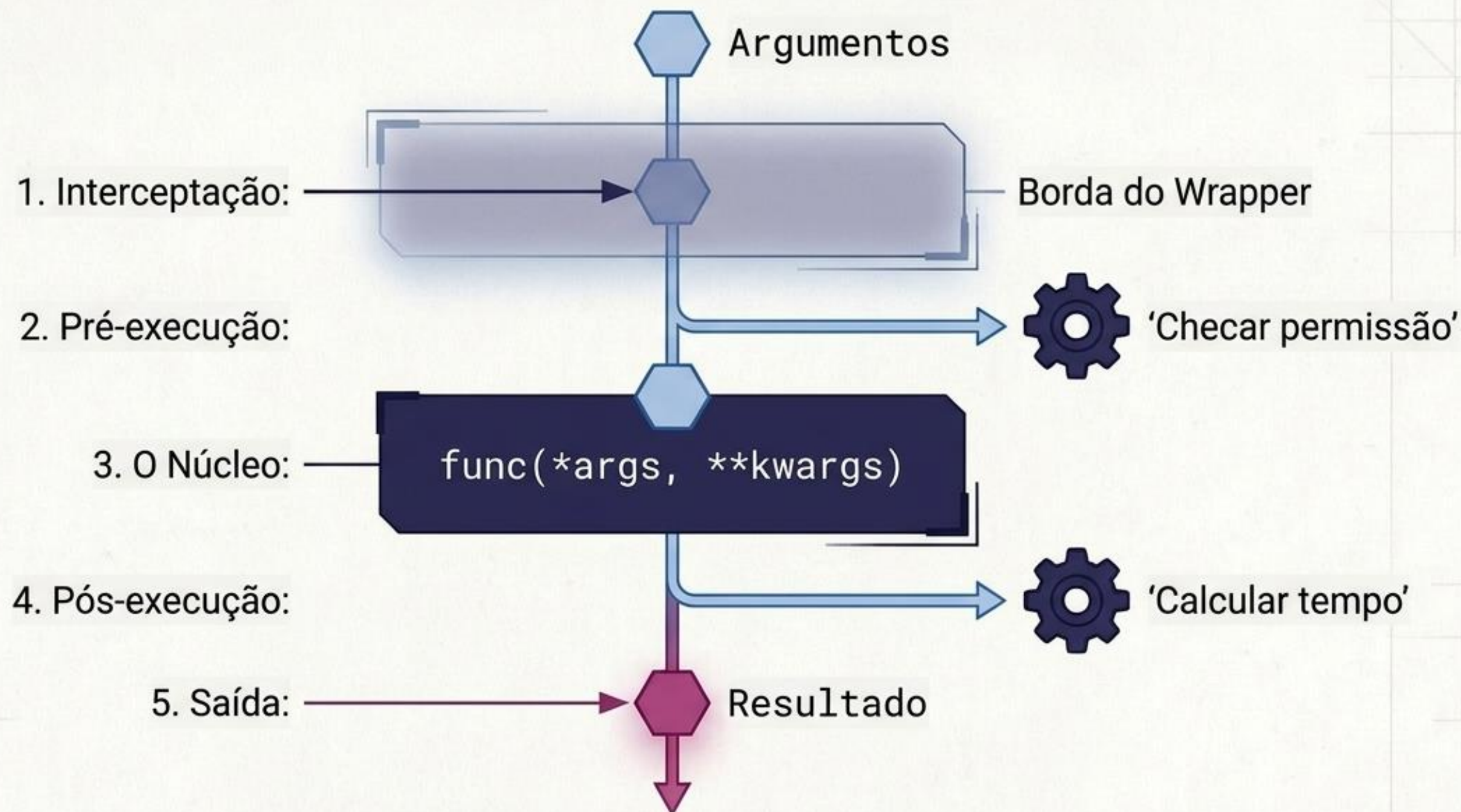
```
def meu_decorador(func):  
    def wrapper(*args, **kwargs):  
        # Pré-execução  
        resultado = func(*args, **kwargs)  
        # Pós-execução  
        return resultado  
  
    return wrapper
```

Camada 1: Função Externa
(Recebe o alvo)

Camada 2: Função Invólucro
(Injeta comportamento e intercepta argumentos)

Camada 3: O Retorno
(Substitui a função original)

Raio-X da Execução: O Ciclo de Vida



Aplicação Prática 1: Telemetria e Tempo de Execução

Ideal para monitoramento e profiling. O decorador injeta o cálculo de tempo e logging ao redor da função original de forma invisível, mantendo a regra de negócio central imaculada.

Code Definition

```
@timer -----  
def processar_dados_pesados(dataset):  
    # Processamento intensivo  
    return dataset.clean()
```

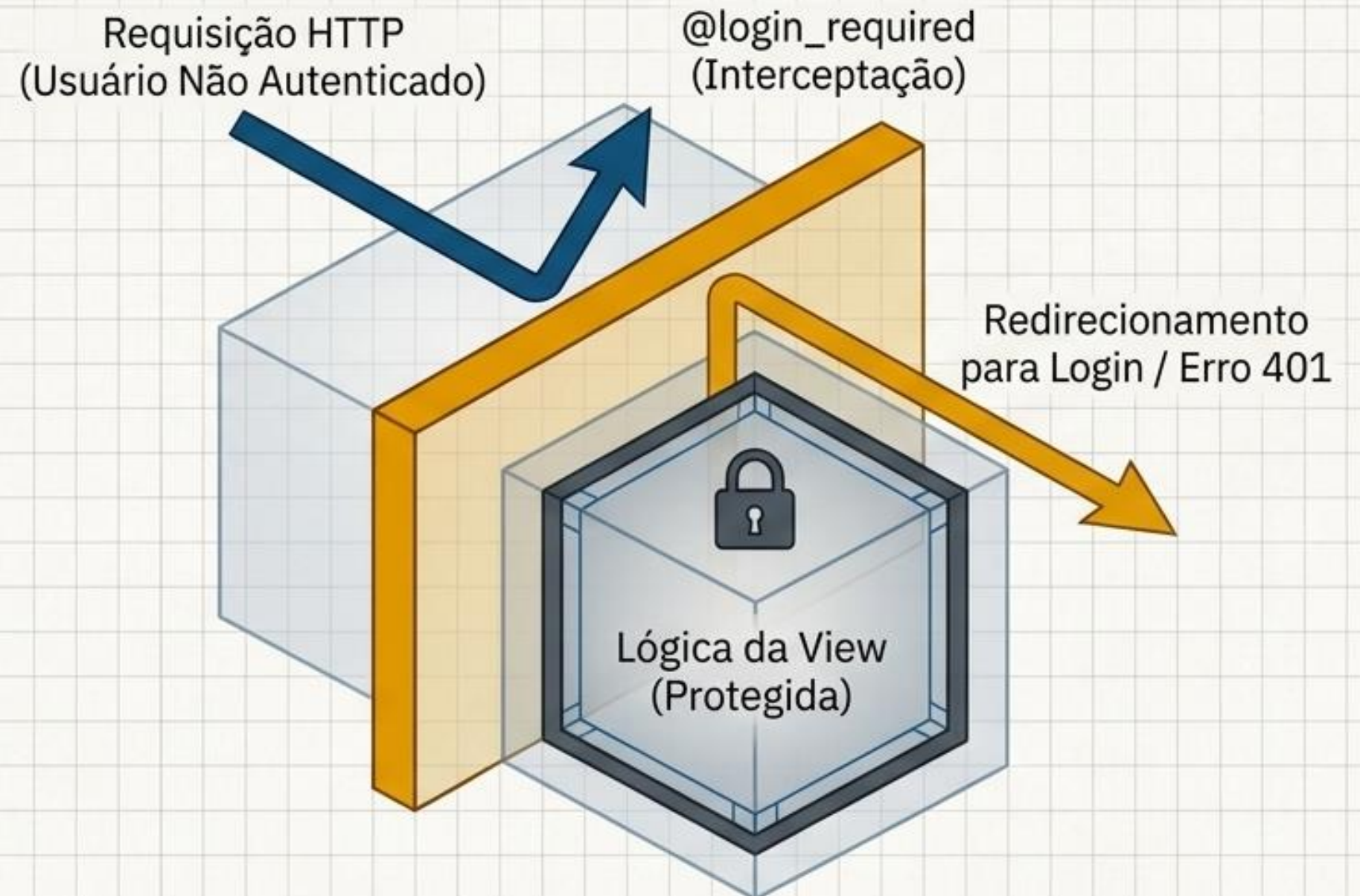
Output Simulation

> Executando script...
> [LOG] processar_dados_pesados executada em 2.344 segundos.
> Processamento concluído com sucesso.

Aplicação Prática 2: Validação e Segurança (Web Frameworks)

Frameworks web modernos dependem pesadamente de decoradores. Eles atuam como "guarda-costas" interceptando requisições HTTP para validar tokens, permissões ou roteamento antes de acionar a lógica da view.

```
@app.route('/admin/dashboard')  
@login_required  
def admin_dashboard():  
    return 'Dados Confidenciais'
```



Metaprogramação em POO: Decoradores Nativos do Python

Sustentados pelo Descriptor Protocol do Python, esses decoradores embutidos são essenciais para modelagem avançada de objetos.

Decorator	Propósito	Modificador
@property	Encapsulamento (Getters/Setters)	Transforma método em acesso de atributo. Oculta a execução da função.
@classmethod	Operações a nível de classe	Recebe a classe (cls) implicitamente, não a instância. Útil para construtores alternativos.
@staticmethod	Funções utilitárias isoladas	Não recebe self nem cls. Funciona como função normal dentro do namespace da classe.

O Poder do **@property**: Encapsulamento Elegante

Controle absoluto sob o capô. Permite injetar validações complexas na leitura, escrita ou deleção de atributos, mantendo uma interface limpa de atribuição direta para o usuário da classe. Evita a quebra de contratos de API.

Class Implementation

```
class Retangulo:
    @property
    def width(self):
        return self._width

    @width.setter
    def width(self, value):
        if value <= 0:
            raise ValueError('Inválido')
        self._width = value
```



Execution Simulation

```
obj = Retangulo()
obj.width = 10  # Sucesso silencioso

obj.width = -5  # Aciona a validação:
ValueError: Inválido
```

Síntese Arquitetural: Código Comum vs. Código Decorado

O design de software moderno favorece a separação de responsabilidades (Separation of Concerns). Decoradores forçam a modularidade, transformando lógicas intrincadas em blocos de montar reutilizáveis.

Anti-pattern: Código Monolítico e Desorganizado

```
def processar_pedido(dados, usuario_logado):  
    # 1. Validação de tipos  
    if not isinstance(dados, dict) or 'id' not in dados:  
        raise TypeError("Dados inválidos")  
  
    # 2. Logging de acesso  
    print(f"Iniciando processamento para usuário: {usuario_logado}")  
    logger.info(f"Acesso ao endpoint processar_pedido por {usuario_logado}")  
  
    # 3. Gerenciamento de transação e banco de dados  
    db_connection = get_db()  
    transaction = db_connection.begin()  
    try:  
        # 4. Lógica de negócios  
        pedido = db_connection.find_pedido(dados['id'])  
        if not pedido:  
            raise ValueError("Pedido não encontrado")  
  
        total = calcular_total(pedido)  
        aplicar_desconto(pedido, total)  
        db_connection.update_pedido(pedido)  
  
        transaction.commit()  
        print("Pedido processado com sucesso.")  
        return {"status": "sucesso", "pedido_id": pedido.id}  
    except Exception as e:  
        transaction.rollback()  
        print(f"Erro ao processar: {e}")  
        logger.error(f"Falha na transação: {e}")  
        raise e  
    finally:  
        db_connection.close()
```

Pythonic: Código Modular e Organizado

@valida_tipo

@log_acesso

@db_transaction

```
def processar_pedido(dados, usuario_logado):  
    # Lógica de negócios pura  
    pedido = find_pedido(dados['id'])  
    total = calcular_total(pedido)  
    aplicar_desconto(pedido, total)  
    update_pedido(pedido)  
    return {"status": "sucesso", "pedido_id": pedido.id}
```

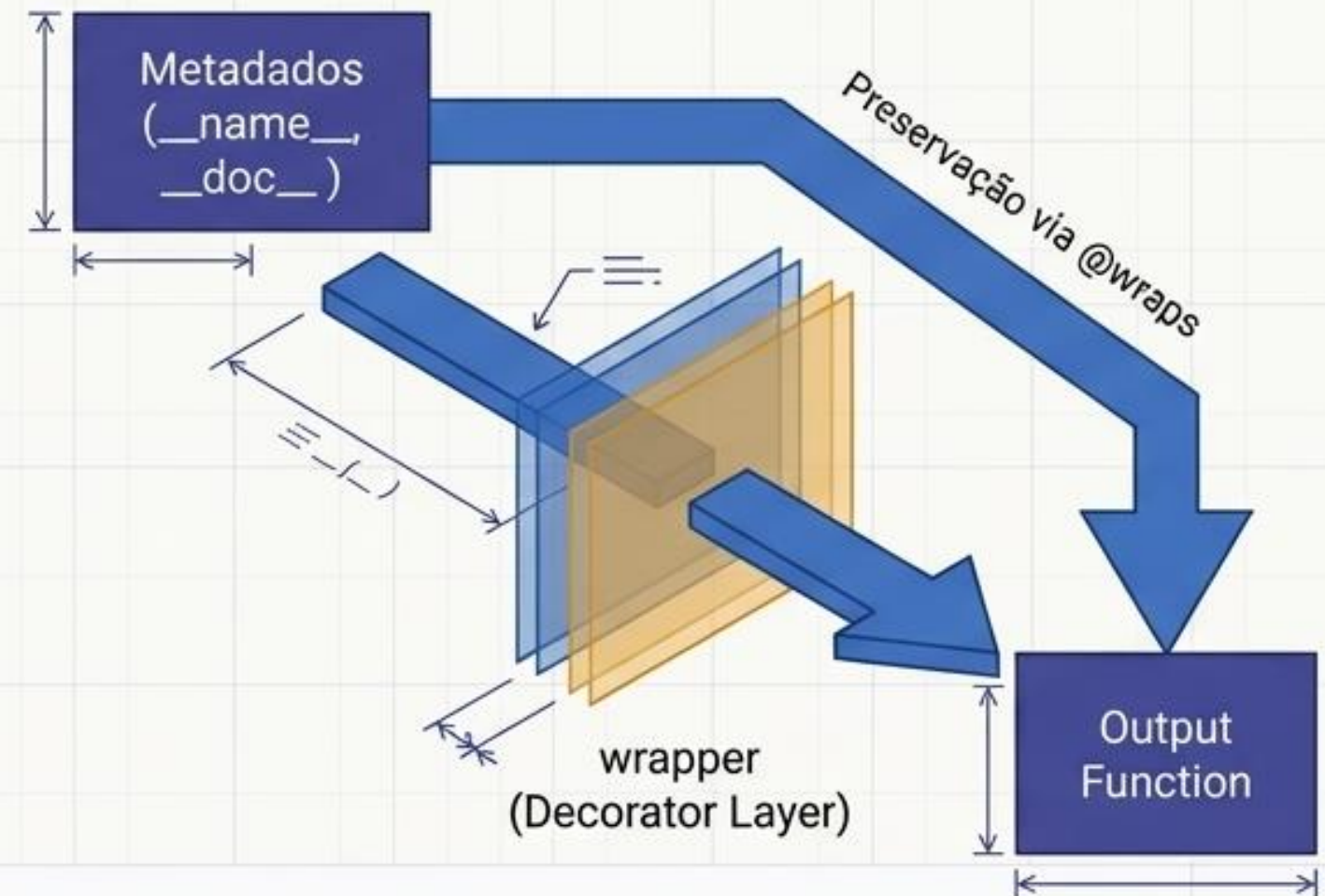
Melhores Práticas: A Regra de Ouro (functools.wraps)

Sem o uso de @wraps, a função original perde sua identidade e metadados, assumindo o nome e a docstring do "wrapper". Sempre importe e utilize functools.wraps para preservar a integridade da sua função para debuggers e documentação!



```
from functools import wraps

def meu_decorador(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```



Da Sintaxe à Arquitetura de Sistemas

Dominar decoradores não é apenas decorar uma sintaxe; é compreender o modelo de dados subjacente do Python. Eles formam a ponte essencial entre scripts básicos e a engenharia de software de sistemas avançados e escaláveis.

